Experiment 5

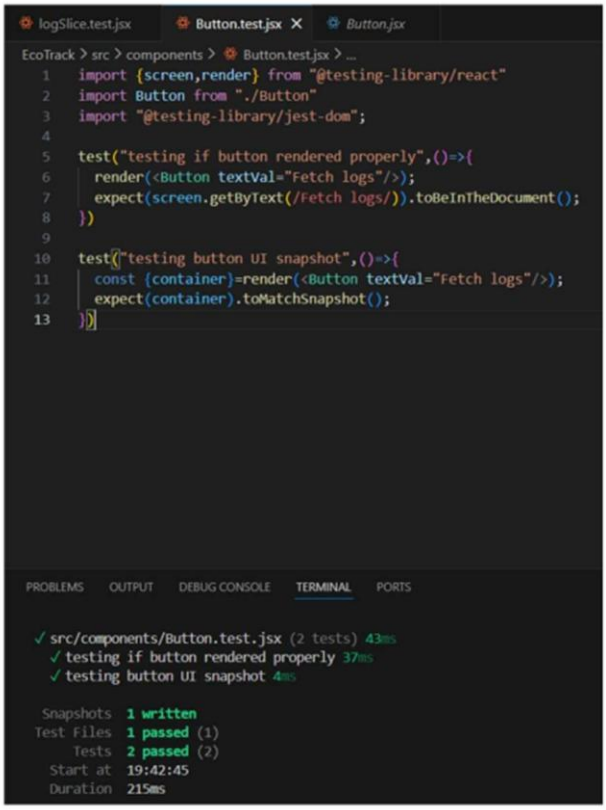
Student Name: Divyansh Manhas UID: 23BAI70239
Branch: AIT-CSE Section: 23AIT_KRG-1 A
Semester: 6 Date of Performance: 13/02/2026
Subject Name: Full Stack II Subject Code: 23CSH-382

- Aim:
  The aim of this experiment is to verify the correctness, stability, and reliability of the EcoTrack React application by implementing automated testing techniques including unit testing, snapshot testing, component rendering tests, Redux reducer testing, and mocking using Jest, React Testing Library, and Redux Mock Store.
- Objectives:
  The primary objectives of this experiment are:
  1. To understand the importance of automated testing in modern frontend applications.
  2. To write unit tests for JavaScript logic using Jest.
  3. To validate UI rendering using React Testing Library.
  4. To perform snapshot testing to detect unintended UI structure changes.
  5. To test Redux reducers independently for correct state transitions.
  6. To mock Redux store behavior using redux-mock-store.
  7. To simulate application state for component-level testing.
  8. To verify loading and state transition logic in asynchronous Redux flows.
  9. To analyze test failures systematically and debug errors effectively.
  10. To improve software maintainability and confidence through test coverage.

- Implementation:
  The testing phase of the EcoTrack application was implemented through the following structured steps:

1. Component Rendering Test (Button Component) o The custom Button component was tested to verify whether it renders correctly with given props.
    - The component was rendered using render() from React Testing Library.
    - The DOM was queried using screen.getByText().
    - The presence of the button text was validated using Jest-DOM o matchers.
2. Snapshot Testing o Snapshot testing was implemented to capture the UI structure of the Button component and detect unintended structural changes in future updates.
    - The rendered container was captured. o toMatchSnapshot() was used to generate and compare snapshot files.
3. Redux Mock Store Testing (Logs Component) o To test the Logs component without relying on the real store, a mock Redux store was created using redux-mock-store.
    - Thunk middleware was included to allow dispatching async actions. o A predefined state was injected into the store. o The component was wrapped inside <Provider>. o This ensured the UI correctly rendered based on mocked Redux state.
4. Reducer Unit Testing
   Retrieved The Redux reducer (logSlice) was tested independently to validate:
    - Initial state correctness
    - Proper state transition during async pending state
    - These tests confirmed correct reducer behavior during asynchronous lifecycle stages.

- Output:



- Results:
  The EcoTrack application was successfully validated using automated testing techniques. Component-level tests ensured correct UI rendering, snapshot testing protected against unintended structural modifications, and Redux reducer tests verified accurate state transitions. Mock store testing confirmed that components behave correctly under simulated application states. This experiment strengthened application reliability, reduced dependency on manual testing, and improved overall code confidence and maintainability.

- Learning Outcomes:
  After completing this experiment, I have learnt to:

1. Write unit tests for React components using React Testing Library.
2. Perform snapshot testing to track UI changes.
3. Use Jest matchers to validate expected outputs.
4. Test Redux reducers independently for state correctness.